Module 9 Lab Session 1: Centralized State with SignalStore

Story Thread

It is Enrollment Week at CoTBE. Two thousand students are registering simultaneously, and Dawit is working through the queue from his instructor dashboard.

He opens the **Enrollment List** widget on the left panel and clicks **Approve** next to Liya's enrollment. The status flips to "Approved" immediately. He then switches to the **Dashboard Summary** widget on the right panel to check remaining pending counts.

The summary still says **42 Pending**.

He refreshes the entire page. Now it reads **41 Pending**.

What happened? Both widgets called `this.http.get('/api/enrollments').subscribe()` independently when they initialized. The Enrollment List updated its local `signal()` when the approve call completed, but the Dashboard Summary was running on its own copy of the data , a completely separate signal that knew nothing about the approval.

This is the core problem of **state drift**: when multiple components independently fetch and manage copies of the same mutable data, they inevitably fall out of sync.

The solution is a **single source of truth**. Instead of each widget holding its own signal, both read from one centralized store. When the store changes, every widget that reads from it updates automatically, no refresh, no extra API calls, no drift.

Prerequisites: Domain Model and API Service

Before building the store, you need the enrollment data contract and the HTTP service that talks to the .NET API.

The Enrollment Model

Create `src/app/models/enrollment.model.ts`:

```typescript
export interface Enrollment {
  id: string;
  studentId: number;
  studentName: string;
  courseId: number;
  courseName: string;
  status: 'Pending' | 'Approved' | 'Rejected';
  enrolledAt: string;
}
```

The Enrollment Service

Generate the service (`ng generate service services/enrollment`) and implement the API client:

```typescript
import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Enrollment } from '../models/enrollment.model';

@Service()
export class EnrollmentService {
  private http = inject(HttpClient);
  private baseUrl = 'http://localhost:5000/api/enrollments';

  getAll(): Observable<Enrollment[]> {
    return this.http.get<Enrollment[]>(this.baseUrl);
  }

  approve(id: string): Observable<void> {
    return this.http.post<void>(`${this.baseUrl}/${id}/approve`, {});
  }
}
```

The base URL uses a relative path (`/api/enrollments`). In M10 Session 1, you will configure the Angular dev proxy and environment files to route this to your .NET API on port 5000. For now, this relative path keeps your service code deployment-ready from day one.

Exercise 1: Centralized State with NgRx SignalStore

Why Local Signals Drift

In Module 8, you learned that `signal()` creates a reactive value that components can read and write. But here is the catch: each `signal()` call creates an **independent, isolated** reactive container. If two components each create their

own enrollment signal or each call `http.get()` separately; they hold separate copies of the data. Changing one copy does not affect the other.

This is fine for UI-local state (a sidebar toggle, a selected tab index). But for shared mutable data like enrollment records that multiple widgets display and modify, you need a **singleton store ;** one instance in memory, shared by every component that injects it.

Install NgRx SignalStore

Open a terminal in your Angular project and install the package:

```
npm install @ngrx/signals
```

Build the Enrollment Store

Create `src/app/store/enrollment.store.ts`. Read the inline comments carefully, each building block solves a specific architectural problem:

```
import { computed, inject } from '@angular/core';
import {
  signalStore,
  withComputed,
  withMethods,
  patchState,
  withState,
} from '@ngrx/signals';
import {
  withEntities,
  setAllEntities,
  updateEntity,
} from '@ngrx/signals/entities';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { pipe, concatMap, tap, catchError, EMPTY } from 'rxjs';
import { EnrollmentService } from '../services/enrollment.service';
import { Enrollment } from '../models/enrollment.model';

export const EnrollmentStore = signalStore(
  { providedIn: 'root' },

  // withState adds simple properties alongside the entity collection
  withState({ isLoading: false, error: null as string | null }),

  // withEntities creates an O(1) ID-indexed dictionary for the enrollm
ent collection.
  // Internally, it stores { ids: string[], entityMap: Record<string, E
nrollment> }
  // so lookups and updates by ID are instant — no array scanning.
```

```
  withEntities<Enrollment>(),

  // withComputed creates read-only derived signals that update automat
ically.
  // pendingCount recalculates every time the entity collection changes.
  withComputed((store) => ({
    pendingCount: computed(
      () => store.entities().filter(e => e.status === 'Pending').length
    ),
  })),

  withMethods((store, api = inject(EnrollmentService)) => ({

    //  Loading Data
    // Why concatMap here? Because concatMap processes one emission at
a time
    // in strict order. If something triggers loadEnrollments() twice q
uickly,
    // concatMap waits for the first HTTP response before starting the
second.
    // switchMap would cancel the first request (data loss risk).
    // mergeMap would run both in parallel (race condition risk).
    loadEnrollments: rxMethod<void>(
      pipe(
        tap(() => patchState(store, { isLoading: true, error: null })),
        concatMap(() =>
          api.getAll().pipe(
            tap(rows => patchState(store, setAllEntities(rows), { isLoa
ding: false })),
            catchError(err => {
              patchState(store, { isLoading: false, error: err.message
 });
              return EMPTY; // EMPTY completes silently so the rxMethod
 pipeline survives
            })
          )
        )
      )
    ),

    //  Optimistic Approve
    // Step 1: Instantly flip the status to "Approved" in the store.
    //         Every component reading from the store sees the change i
mmediately.
    // Step 2: Send the approval to the server.
    // Step 3: If the server rejects it, roll back the status to "Pendi
ng."
    approveEnrollment: rxMethod<string>(
      pipe(
```

```
        tap(id => {
          // Optimistic update — the UI reacts before the network round
-trip completes
          patchState(store, updateEntity({ id, changes: { status: 'Appr
oved' } }));
        }),
        concatMap(id =>
          api.approve(id).pipe(
            catchError(err => {
              // Server said no — restore the previous state
              patchState(store, updateEntity({ id, changes: { status: '
Pending' } }));
              patchState(store, { error: 'Server rejected the approval.
 Check enrollment constraints.' });
              return EMPTY;
            })
          )
        )
      )
    ),
  }))
);
```

Wire the Store into Your Component

Generate the enrollment list component (`ng generate component features/enrollment-list`), then connect it to the store:

```
import { Component, inject, OnInit } from '@angular/core';
import { EnrollmentStore } from '../../store/enrollment.store';

@Component({
  selector: 'tms-enrollment-list',
  standalone: true,
  templateUrl: './enrollment-list.component.html'
})
export class EnrollmentListComponent implements OnInit {
  store = inject(EnrollmentStore);

  ngOnInit() {
    this.store.loadEnrollments();
  }

  onApprove(id: string) {
    this.store.approveEnrollment(id);
  }
}
```

In the template, bind to the store's entity collection:

```
@if (store.isLoading()) {
  <p>Loading enrollments...</p>
}

@for (enrollment of store.entities(); track enrollment.id) {
  <div class="enrollment-card">
    <span>{{ enrollment.studentName }} — {{ enrollment.courseName }}</s
pan>
    <span class="status">{{ enrollment.status }}</span>
    @if (enrollment.status === 'Pending') {
      <button (click)="onApprove(enrollment.id)">Approve</button>
    }
  </div>
}

@if (store.error()) {
  <p class="error">{{ store.error() }}</p>
}
```

What You Just Built

To see the store in action, open two components that both read from `EnrollmentStore` the enrollment list and a dashboard summary widget showing `store.pendingCount()`. Click **Approve** on Liya's enrollment in the list. Without navigating away or refreshing, the dashboard widget's pending count drops by one automatically.

This works because both components inject the same singleton store. The moment `patchState` fires, the store's entity dictionary updates, and every component bound to `store.entities()` or `store.pendingCount()` re-renders automatically. No manual refresh, no duplicate API calls, no state drift.

Cross-tab sync (two separate browser windows updating simultaneously) requires a real-time push channel. You will build that in Session 3 using SignalR.

This is the architectural foundation for everything that follows in M9: performance optimization, enterprise grids, defensive RxJS, and real-time sync all build on top of this centralized store.

What's Next

Keep this project running. In **Session 2**, you will optimize the Command Center for slow 3G connections using `@defer` blocks and replace the basic HTML list with a sortable, paginated Angular Material grid both wired directly to this same `EnrollmentStore`.